```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

"""
CANSAT FLIGHT SOFTWARE
Telemetría CSV por LoRa
OLED 3 pantallas
NeoPixel estados
Buzzer StarWars + modo búsqueda
1 Hz
"""

import time
import serial
from datetime import datetime, timezone, timedelta

from cansat_sensors import create_default_sensor_manager

# OLED
from luma.core.interface.serial import spi
from luma.oled.device import sh1106
from luma.core.render import canvas
from PIL import ImageFont

# LED
import board
import neopixel

# BUZZER
import RPi.GPIO as GPIO


# ---------------------------------------------------
# OLED CONFIG
# ---------------------------------------------------

serial_oled = spi(device=0, port=0, gpio_DC=13, gpio_RST=6)
device = sh1106(serial_oled)

font = ImageFont.load_default()

OLED_OFFSET = 3


# ---------------------------------------------------
# LORA UART
# ---------------------------------------------------

lora = serial.Serial(
    "/dev/serial0",
    baudrate=9600,
    timeout=1
)


# ---------------------------------------------------
# NEOPIXEL
# ---------------------------------------------------

PIXEL_PIN = board.D18
NUM_PIXELS = 1

pixels = neopixel.NeoPixel(
    PIXEL_PIN,
    NUM_PIXELS,
    brightness=0.3,
    auto_write=False
)
```

```


# ---------------------------------------------------
# BUZZER
# ---------------------------------------------------

BUZZER_PIN = 24

GPIO.setmode(GPIO.BCM)
GPIO.setup(BUZZER_PIN, GPIO.OUT)

buzzer = GPIO.PWM(BUZZER_PIN, 440)


# ---------------------------------------------------
# NOTAS MUSICALES
# ---------------------------------------------------

A4=440
A4S=466
B4=494
C5=523
D5=587
E5=659
F5=698
G5=784
A5=880


# ---------------------------------------------------
# LED STATES
# ---------------------------------------------------

def led_boot():

    pixels[0]=(255,0,0)
    pixels.show()
    time.sleep(0.3)

    pixels[0]=(0,0,0)
    pixels.show()
    time.sleep(0.3)


def led_tx():

    pixels[0]=(0,255,0)
    pixels.show()
    time.sleep(0.05)

    pixels[0]=(0,0,0)
    pixels.show()


def led_error():

    pixels[0]=(255,0,0)
    pixels.show()


def led_sensor_warning():

    pixels[0]=(255,120,0)
    pixels.show()


# ---------------------------------------------------
# BUZZER
# ---------------------------------------------------
```

```


def tone(freq, duration):

    buzzer.ChangeFrequency(freq)
    buzzer.start(70)

    time.sleep(duration)

    buzzer.stop()

    time.sleep(0.02)


# --------------------------------------------------
# STAR WARS MELODY
# --------------------------------------------------


def star_wars():

    melody=[

        (A4,0.5),(A4,0.5),(A4,0.5),
        (F5,0.35),(C5,0.15),

        (A4,0.5),(F5,0.35),(C5,0.15),(A4,0.9),

        (E5,0.5),(E5,0.5),(E5,0.5),
        (F5,0.35),(C5,0.15),

        (G5,0.5),(F5,0.35),(C5,0.15),(A4,0.9)

    ]

    for note,duration in melody:

        tone(note,duration)


# --------------------------------------------------
# SEARCH MODE
# --------------------------------------------------


def search_beacon():

    while True:

        star_wars()

        time.sleep(20)


# --------------------------------------------------
# UNIDADES PARA CABECERA
# --------------------------------------------------

UNITS={

"time_s":"s",

"temperature_C":"C",
"pressure_hPa":"hPa",
"humidity_pct":"%",

"altitude_barometric_m":"m",

"quat_w":"-",
```

```
"quat_x":"-",
"quat_y":"-",
"quat_z":"-",

"accel_x":"m/s2",
"accel_y":"m/s2",
"accel_z":"m/s2",

"gps_fix":"bool",
"satellites":"n",

"latitude":"deg",
"longitude":"deg",

"gps_altitude":"m",
"gps_speed_mps":"m/s",
"gps_speed_kmh":"km/h",
"gps_course":"deg",

"soil_raw":"adc",
"soil_humidity_pct":"%",
"soil_state":"cat"

}


# --------------------------------------------------
# HORA LOCAL
# --------------------------------------------------


def local_time():

    tz = timezone(timedelta(hours=1))

    return datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")


# --------------------------------------------------
# CSV
# --------------------------------------------------


def build_header(keys):

    header=["packet"]

    for k in keys:

        unit=UNITS.get(k,"")

        header.append(f"{k}({unit})")

    return ",".join(header)



def build_csv(packet,data):

    row=[str(packet)]

    for k in data.keys():

        row.append(str(data[k]))

    return ",".join(row)


# --------------------------------------------------
```

```
# OLED PANTALLAS
# ---------------------------------------------------


def oled_boot():

    with canvas(device) as draw:

        draw.text((0,0+OLED_OFFSET),"TEAM ROCKET",font=font,fill=255)
        draw.text((0,15+OLED_OFFSET),"CANSAT START",font=font,fill=255)
        draw.text((0,35+OLED_OFFSET),"INIT SYSTEM",font=font,fill=255)



def oled_screen_status(packet,data):

    gps="FIX" if data.get("gps_fix",False) else "NO"

    t=data.get("time_s",0)

    with canvas(device) as draw:

        draw.text((0,0+OLED_OFFSET),"PKT:",font=font,fill=255)
        draw.text((60,0+OLED_OFFSET),str(packet),font=font,fill=255)

        draw.text((0,15+OLED_OFFSET),"GPS:",font=font,fill=255)
        draw.text((60,15+OLED_OFFSET),gps,font=font,fill=255)

        draw.text((0,30+OLED_OFFSET),"TIME:",font=font,fill=255)
        draw.text((60,30+OLED_OFFSET),f"{t:.1f}",font=font,fill=255)

        draw.text((0,45+OLED_OFFSET),"TX ACTIVE",font=font,fill=255)



def oled_screen_science(data):

    alt=data.get("altitude_barometric_m",0)
    temp=data.get("temperature_C",0)
    vel=data.get("gps_speed_kmh",0)

    with canvas(device) as draw:

        draw.text((0,0+OLED_OFFSET),"ALT:",font=font,fill=255)
        draw.text((60,0+OLED_OFFSET),f"{alt:.1f}",font=font,fill=255)

        draw.text((0,15+OLED_OFFSET),"TEMP:",font=font,fill=255)
        draw.text((60,15+OLED_OFFSET),f"{temp:.1f}",font=font,fill=255)

        draw.text((0,30+OLED_OFFSET),"VEL:",font=font,fill=255)
        draw.text((60,30+OLED_OFFSET),f"{vel:.1f}",font=font,fill=255)

        draw.text((0,45+OLED_OFFSET),"SCIENCE",font=font,fill=255)



def oled_screen_radio(packet):

    with canvas(device) as draw:

        draw.text((0,0+OLED_OFFSET),"LORA RADIO",font=font,fill=255)

        draw.text((0,20+OLED_OFFSET),"SENDING PKT",font=font,fill=255)

        draw.text((0,40+OLED_OFFSET),str(packet),font=font,fill=255)


# ---------------------------------------------------
# MAIN
```

```
# ------------------------------------------------


def main():

    for i in range(6):

        led_boot()

    oled_boot()

    time.sleep(2)

    sensors=create_default_sensor_manager()

    sensors.init_all()

    csv_file="cansat_data.csv"

    packet=0

    screen=0

    last_screen=time.time()

    header_sent=False

    mission_start_sent=False

    landing_counter=0

    landed=False

    search_mode=False


    while True:

        data=sensors.read_all()

        sensors.save_csv(csv_file,data)


        # SENSOR CHECK

        sensor_error=False

        for v in data.values():

            if v is None:

                sensor_error=True

        if sensor_error:

            led_sensor_warning()


        # MISSION START

        if not mission_start_sent:

            start=local_time()

            lora.write((f"MISSION_START,{start}\n").encode())

            mission_start_sent=True


```

```
        # HEADER

        if not header_sent:

            header=build_header(data.keys())

            lora.write((header+"\n").encode())

            header_sent=True


        # CSV

        line=build_csv(packet,data)

        lora.write((line+"\n").encode())

        led_tx()


        # LANDING DETECTION

        vel=data.get("gps_speed_mps",0)

        if vel<0.5:

            landing_counter+=1

        else:

            landing_counter=0


        if landing_counter>5 and not landed:

            star_wars()

            landed=True

            search_mode=True


        if search_mode:

            search_beacon()


        # OLED ROTATION

        if time.time()-last_screen>2:

            screen=(screen+1)%3

            last_screen=time.time()


        if screen==0:

            oled_screen_status(packet,data)

        elif screen==1:

            oled_screen_science(data)

        else:

            oled_screen_radio(packet)


```

```
        packet+=1


        time.sleep(1)


if __name__=="__main__":

    try:

        main()

    except KeyboardInterrupt:

        print("CANSAT STOP")

        GPIO.cleanup()

```